P1 Handoff Report

Entities, repositories, migrations, and seed data for the PRBS backend — built by P2 to unblock Phase 5/6/7 testing.

Branch	p1/entities-for-p2
App location	prbs-backend/ (sibling of prbs-app/)
Stack	Spring Boot 3.2.5, Java 17, JPA, Flyway, H2 + Postgres
Status	All 44 tests passing; runnable via mvn spring-boot:run
Author of P1 code	P2 (this is a handoff, not the final P1 deliverable)

Why this report exists

P2's deliverables (auth endpoints, booking notifications, reminder scheduler) all depend on entities, repositories, and database schema that are formally P1's responsibility. Rather than block until P1 ships, I built the **minimum viable P1** needed to make Phase 5/6/7 testable end-to-end. All of that code lives in `prbs-backend/` on branch `p1/entities-for-p2`.

This report is for you, the actual P1 owner. The intent is not to claim P1's territory — you should treat what's here as a starting point: extend, refactor, or rewrite as you see fit. The shapes were chosen to match the data model in the React prototype (`prbs-app/src/data.js`) so frontend integration stays straightforward.

TL;DR

What I built: 5 JPA entities, 5 repositories, 6 Flyway migrations (schema + seeded users), a `BookingService` skeleton with email notification hooks, and H2/Postgres configuration. 44 tests passing.

What's still yours: the REST controllers for bookings, availability, users (admin), and settings. Business rules around bookings (slot conflicts, cancellation window enforcement). Any schema evolutions you'd prefer.

Architecture & file layout

`prbs-backend/` is a standalone Spring Boot app. It boots against H2 in-memory by default (no DB setup), and switches to Postgres with `--spring.profiles.active=postgres`.

```
prbs-backend/
■■■ pom.xml                          Spring Boot 3.2.5, JPA, Flyway, jjwt, Lombok
■■■ README.md                        How to run + env-var checklist
■■■ .env.example                     Mail provider template (Outlook/Gmail/SendGrid)
■■■ postman/                         Postman collection + setup guide
■■■ src/
    ■■■ main/
        ■■■ java/com/auca/prbs/
        ■   ■■■ PrbsBackendApplication.java
        ■   ■■■ entity/              <-- P1: User, OtpToken, Settings, Booking,
        ■   ■                            Availability + 3 enums
        ■   ■■■ repository/          <-- P1: 5 Spring Data JPA repos
        ■   ■■■ security/            P2: JWT filter, SecurityConfig, rate-limit,
        ■   ■                            revocation store
        ■   ■■■ service/             P1+P2: OtpService, EmailService,
        ■   ■                            AuthService, BookingService,
        ■   ■                            ReminderScheduler
        ■   ■■■ controller/          P2: AuthController only
        ■   ■■■ dto/                 P2: Request/response envelopes
        ■   ■■■ exception/           P2: Typed errors + global handler
        ■■■ resources/
            ■■■ application.yml      <-- P1: datasource, jpa, flyway, mail
            ■■■ db/migration/        <-- P1: V1..V6 SQL
```

Entities

Each entity uses `BIGINT GENERATED BY DEFAULT AS IDENTITY` for primary keys — standard SQL, works on both H2 (in PostgreSQL mode) and real Postgres without dialect-specific tweaks. Audit columns (`created_at`, `updated_at`) are set in `@PrePersist` / `@PreUpdate` hooks, not via DB defaults — keeps the JPA layer the source of truth.

User

Field	Type	Notes
id	Long (IDENTITY)	PK
name	String	Display name for emails
email	String	UNIQUE; lookup key for /auth/send-otp
role	UserRole enum	STUDENT, SUPERVISOR, ADMIN
status	UserStatus enum	ACTIVE, INACTIVE (inactive users are 403'd)
created_at, updated_at	Instant	audit

OtpToken

Replaces P2's stub POJO. The `attempts` column is what enables the brute-force lockout in `OtpService` (locks at 5 failures).

Field	Type	Notes
id	Long (IDENTITY)	PK
user_id	Long (FK → users)	Which user this code belongs to
token_hash	String (100)	BCrypt hash; plain code only sent in email
expires_at	LocalDateTime	Set from settings.otp_expiry minutes
used	boolean	True after successful verify; prevents replay
attempts	int	Failed verifies. Token locks at MAX_ATTEMPTS=5
created_at	Instant	audit

Settings (singleton)

One row, id=1, seeded by migration V3. All values are in minutes, matching `MOCK_SETTINGS_INIT` in the React prototype.

Field	Type	Default	Read by
otp_expiry	int	10	AuthService.sendOtp
cancel_window	int	60	(not yet enforced — P3 booking logic)
reminder_time	int	30	ReminderScheduler

Booking

I deliberately combined *date* and *time* into a single `slot_at LocalDateTime` column. The React prototype exposes them separately, but a single column makes the reminder query trivially expressible as a BETWEEN. The API DTOs can still serialize as separate date/time fields if you prefer.

Field	Type	Notes
id	Long (IDENTITY)	PK
student_id	Long (FK → users)	
supervisor_id	Long (FK → users)	Used to address the supervisor alert email
name	String	Snapshot of student name (for emails)
group_number	int	Matches MOCK_BOOKINGS_INIT.group
project	String (200)	
slot_at	LocalDateTime	Single column; date + time combined
status	BookingStatus enum	CONFIRMED, CANCELLED, COMPLETED, NO_SHOW
meet_url	String (500)	
reminder_sent	boolean	Set true after the scheduler emails; idempotency
created_at, updated_at	Instant	audit

Availability

Mirrors `MOCK_AVAILABILITY` in the React prototype. P2 doesn't read it; I included it for schema completeness so your future `AvailabilityController` has a place to land.

Field	Type	Notes
id	Long (IDENTITY)	PK
supervisor_id	Long (FK → users)	
date	LocalDate	
start_time, end_time	LocalTime	
duration_minutes	int	Slot length
meet_url	String (500)	
created_at	Instant	audit

Repositories

Spring Data JPA interfaces. Custom queries are derivation-based (no JPQL needed yet) so the names document themselves.

Interface	Custom method	Consumer
UserRepository	findByEmail(String)	AuthService.sendOtp/verifyOtp
UserRepository	existsByEmail(String)	(available; not yet used)
OtpTokenRepository	findFirstByUserIdAndUsedFalseOrder-ByExpiresAtDesc(Long)	OtpService.verifyLatest
SettingsRepository	findById(1L) [JpaRepository]	AuthService, ReminderScheduler
BookingRepository	findByStatusAndReminderSentFalseAnd-SlotAtBetween(...)	ReminderScheduler.sendDueReminders
BookingRepository	findByStudentIdOrderBySlotAtDesc	(student dashboard, when P3 wires it)
BookingRepository	findBySupervisorIdOrderBySlotAtDesc	(supervisor dashboard, when P3 wires it)
AvailabilityRepository	findBySupervisorIdAndDate-GreaterThanEqualOrderByDateAsc	(your availability endpoint, when you wire it)

Flyway migrations

Migrations live in `src/main/resources/db/migration/`. All use portable SQL so the same files run unchanged on H2 (PostgreSQL mode) and Postgres — the only DB-specific syntax is the SQL-standard `BIGINT GENERATED BY DEFAULT AS IDENTITY` for IDs.

Version	File	Contents
V1	users.sql	Users table + email index
V2	otp_tokens.sql	OTP tokens table + (user_id, used, expires_at) composite index
V3	settings.sql	Settings table + singleton row seed (10/60/30 minutes)
V4	bookings.sql	Bookings table + (status, reminder_sent, slot_at) reminder-query index
V5	availability.sql	Availability table
V6	seed_demo_users.sql	8 demo users mirroring MOCK_USERS — including an INACTIVE one for negative

Seeded users (V6)

Picked to match the React prototype's mock data so the frontend can talk to the real backend immediately. **Chidi** is intentionally INACTIVE so the `USER_INACTIVE` code path is exercisable.

Email	Role	Status
alice@university.ac.rw	STUDENT	ACTIVE
james@university.ac.rw	STUDENT	ACTIVE
fatima@university.ac.rw	STUDENT	ACTIVE
kwame@university.ac.rw	STUDENT	ACTIVE
priya@university.ac.rw	STUDENT	ACTIVE
chidi@university.ac.rw	STUDENT	INACTIVE
supervisor@university.ac.rw	SUPERVISOR	ACTIVE
admin@university.ac.rw	ADMIN	ACTIVE

`BookingService` (the seam where P3 plugs in)

P2 needed booking events (creation, cancellation) to fire emails. Rather than build a controller, I put the logic in a service so the email side-effects fire from a single seam — one that your future `BookingController` calls into.

```
@Service
public class BookingService {
```

```
    public Booking createBooking(Booking booking) {
        // ... persist; status=CONFIRMED, reminderSent=false ...
        notifyParticipants(saved, "scheduled");
        return saved;
    }

    public Booking cancelBooking(Long id) {
        // ... flip status=CANCELLED ...
        notifyParticipants(saved, "cancelled");
        return saved;
    }
}
```

Your `BookingController` can be as thin as:

```
@PostMapping  ResponseEntity<Booking> create(@RequestBody Booking b)
{ return ResponseEntity.ok(bookingService.createBooking(b)); }
```

… or you can expand `BookingService` with conflict checks, cancel-window enforcement, etc. The email hooks stay where they are.

How P2 consumes P1's foundation

P2 component	Depends on P1
AuthService.sendOtp	UserRepository.findByEmail, SettingsRepository.findById(1), OtpService (persists OtpTok
AuthService.verifyOtp	UserRepository.findByEmail, OtpTokenRepository.findFirst…
AuthService.refresh / logout	UserRepository.findById, in-memory TokenRevocationStore (P2-owned)
OtpService.generateAndPersist	OtpTokenRepository.save
OtpService.verifyLatest	OtpTokenRepository.findFirst…Desc + save
BookingService email hooks	UserRepository.findById (resolve student + supervisor)
ReminderScheduler	BookingRepository reminder query + SettingsRepository.findById(1)

Test coverage

44 tests pass via `mvn test`. The P1-specific coverage:

Test class	Coverage	Count
UserRepositoryTest	Seed users present, findByEmail returns role+status, inactive flag	4
OtpTokenRepositoryTest	Latest unused token query honors expiry ordering; used tokens skipped	2
BookingRepositoryTest	Reminder query returns only confirmed+not-reminded+in-window; skips the rest	1
OtpServiceTest	Hashed persistence, correct verify marks used, wrong increments attempts, lockout at 5, expired	6
AuthFlowIntegrationTest	End-to-end /auth/send-otp → /verify-otp via real HTTP + GreenMail; 401/403 negative paths	4

What's deliberately still your call

I scoped tightly to what P2 needed. The following are P1 territory and I did not implement them:

- **BookingController** — REST endpoints over `BookingService`. Method bodies are essentially one-liners; the interesting work is the validation/authorization rules around them.
- **AvailabilityController** — create/list/delete supervisor availability blocks. The entity + repository are in place; the controller is empty.
- **UserController / SettingsController** for admin — SecurityConfig already restricts `/api/v1/users/**` and `/api/v1/settings/**` to the ADMIN role; the controllers themselves don't exist yet.
- **Booking conflict detection** — nothing prevents two students from booking the same slot. This is business logic that should live in `BookingService.createBooking`.

- **Cancel-window enforcement** — `settings.cancel_window` is seeded but unused. Should gate `BookingService.cancelBooking`.
- **Comprehensive integration tests for your controllers** — once you write them.
- **Slot generation from Availability** — the React app has `generateSlots()`; the backend equivalent isn't built. Likely lives in `AvailabilityService`.
- **Any schema you'd rather restructure** — e.g. if you'd rather have separate `date`/`time` columns on Booking, or split the `name` snapshot, or drop the `attempts` column. Just write a new migration; the entity changes follow.

How to run this locally

```
cd prbs-backend
export JAVA_HOME="$(/usr/libexec/java_home -v 17)"
mvn test                                  # 44 passing
mvn spring-boot:run                       # app on :8080, H2 in-memory
mvn spring-boot:run -Dspring-boot.run.profiles=postgres   # against Postgres
```

Two reasons JDK 17 is required: Lombok 1.18.30 (which Spring Boot 3.2.5 pins) crashes on Java 22+, and the POM targets `<release>17</release>` anyway. The `README.md` documents the workaround.

Recommendations from P2 to P1

- **1.** Take this as a starting point, not a fait accompli. If any naming, field choice, or schema decision conflicts with how you'd structure P1, rewrite it. The tests will tell you what broke; the consumers in `service/` are the only places that depend on the specific signatures.
- **2.** Add your controllers in a new commit on this same branch (`p1/entities-for-p2`) or branch off it. The naming of the branch is misleading at this point — feel free to rename or merge it to `dev` and start fresh.
- **3.** Plan to delete `prbs-p2-sandbox/` (already done in commit `d98a154`). Everything there now lives in `prbs-backend/`.
- **4.** The two open hardening follow-ups from P2's audit: email enumeration on `/auth/send-otp` still returns 404 for unknown emails (should silently return 200), and there's no per-email rate limit (only per-IP). Both are ~30 minutes of work; can land on this branch or later.
- **5.** Use the Postman collection at `prbs-backend/postman/` to verify the auth flow end-to-end before you start adding endpoints. Auto-captures OTP from Mailpit; auto-captures tokens between requests.

Questions or anything that looks wrong — ping me. — *P2*